DEPENDENSY INJECTION

- Bağimlilik Nedir ?
- Injection Nedir ?

IOC(INVERSION OF CONTROL)

ASP.NET CORE'DA IOC YAPILANMASI

- Built-in IoC Continer
  - *Singleton*
  - *Scoped*
  - *Transient*
- Built-in IoC(IServiceCollection) Çalişma Sistemi
- Asp.Net Core IoC(IServiceCollection) Kullanimi(Add fonksiyonu ile)
- Container'a Nesne Eklerken Nesnenin Consturactor'una Parametre Verme
- Interface İle Regİster Etme
- Container'a Eklenen Nesnelerin Controller'da Kullanimlasi/talep edilmesi(Context)
- Container'a Eklenen Nesnelerin Action'da Kullanimlasi/Talep edilmesi
- Container'a Eklenen Nesnelerin View'de Kullanimlasi/Talep edilmesi

Dependensy Injection

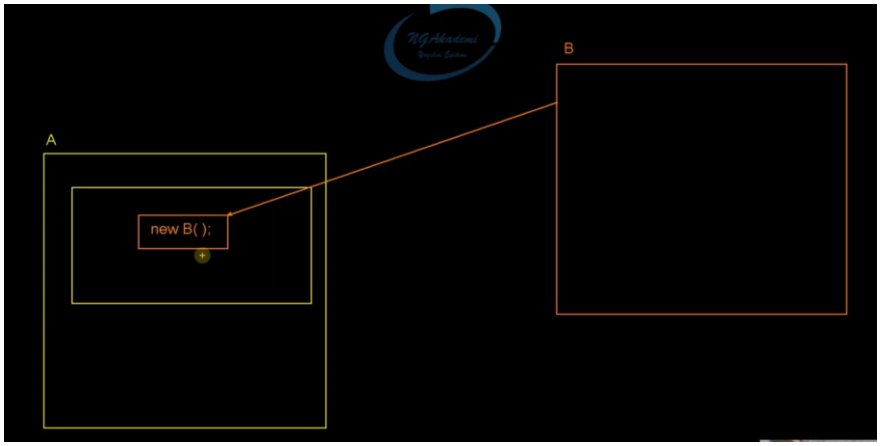
SOLID prensiplerin sonuncu harfi olan Dependecy Inversion yani bağımlılıkların tersine çevrilmesi. Bu yapı bir prensiptir, daha iyi ve verimli kod yazmak için kullanılması gereken bir prensip. Dependecy Inversion prensibini somut olarak uygulanması için Dependency Injection kullanılır.

Yani Depency Injection, bağımlılıkları tersine çevirmek için kullanılan bir Design Pattern'dır.

Asp.Net Core'a özel bir yapı değildir, .Net Core projelerinin tamamında kullanılabilir.

Bağımlılık Nedir ?

Örneğin bir Class A Class'ı var, bu A Class'ında bir methotda B Class'ına ihtiyaç duyuluyor. A Class'ı içerisinde bir B Class'ı oluşturuluyor. Bu işlem sonucunda bir bağımlılık ortaya çıkar. Mesela B Class'ının Consturactor'ında bir değişiklik yapıldığında gidip A Class'ında oluşturulun B Class'ının instance'ını da değiştirmek gerekli.

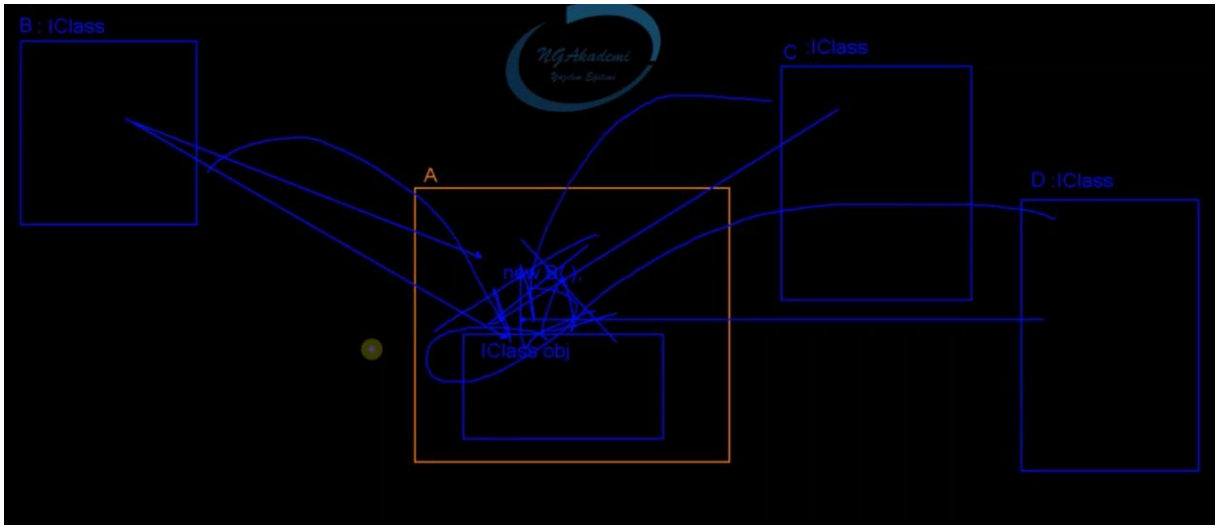


Yani B üzerinde yapılan her değişiklik A'ya yansır. Bu sebeple olabildiğince bağımlılıkları ortadan kaldırmaya çalışırız.

Injection Nedir ?

Örneğin A sınıfının, B sınıfına ihtiyacı var. Burada zaten bir bağımlılık oluştur. Diyelim ki A sınıfının C ve D sınıflarına da ihtiyacı oldu. Bu durumda B sınıfının silinip C sınıfın eklenmesi sonrasında C sınıfının silinip D sınıfın eklenmesi gerekmektedir. Her yapılan operasyonda bu işlem yapılamayacağı için ve her bir durum için farklı bir bağımlılık oluştuğu için dışarıdan bu sınıflar enjekte edilir.

Yani bu sınıfların hepsi IClass'dan türemiş olsun. BU sayede A sınıfı içerisinde sadece IClass kullanılması yeterlidir. IClass artık B,C ve D sınıflarını karşılayabilir.



Özetle bir sınıf içerisinde, başka bir sınıfın Instance'ını "new" operatörü ile oluşturuyorsa. O sınıf ilgili sınıfa bağımlı olur. Bu bağımlılığı ortadan kaldırmak için ilgili sınıf parametreden(Örn. Consturactor parametresi) alınmaya çalışılır. Bu sayede davranış değişse de yani yeni sınıf ihtiyaçları doğsa da kaynak kodu değiştirmeye gerek kalmaz.

Örneğin bir insan Parlement içiyor, bu kişinin sigarası bittiği zaman arkadaşı Marlboro veriyor. Eğer bu kişi Parlement dışında sigara içiyorsa Parlement'e bağımlıdır. Eğer o anki sigara ihtiyacını başka sigara markaları ile giderebiliyorsa bu kişi Parlement'e değil sigaraya bağımlıdır.

Sınıf içerisinde ihtiyaç duyulan nesneyi Consturactor ile veya setter metodu ile parametre olarak alınması gerektiğini savunur Dependecny Injection.

Dependency Injection, bağımlılıkları yok etmek değil soyutlamaktır. Yani sigarayı bırakmıyoruz sadece Parlement'e olan bağımlılığı ortadan kaldırıyoruz.

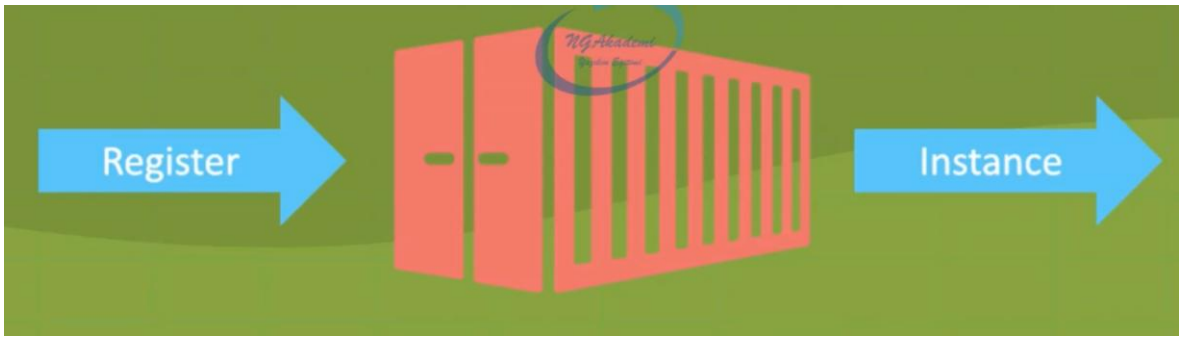
IoC(Inversion Of Control)

Bazı durumlarda bir sınıf çok sayıda farklı sınıfa ihtiyaç duyabilir. Böyle bir durumda her bir sınıf için ayrı ayrı referans vermek yerine bu işlemi otomatik bir şekilde yapan container'lar kullanılır. Bu Container'lara gerekli sınıfların nesneleri verilir ve ihtiyaç halinde bu container üzerinden ilgili nesnelere ulaşılır. Buna Inversion Of Control(Kontrolün ters çevrilmesi) denir.

Örneğin bir A sınıfı B sınıfına bağımlıdır. B sınıfında yapılacak her değişiklik A'yı etkilemektedir. Yani aslında A sınıfının kontrolü B'nin elindedir.

Bir devlet dairesi olsun, bu devlet dairesinde belirli bir işi sadece Ahmet yapabiliyor. Bu durumda daire başkanı bile Ahmet'e bağlıdır çünkü bu işi sadece Ahmet yapabiliyor. Inversion Of Control ile bağımlılık tersine çevrilir ve artık dairede çalışan herkes daire başkanına bağlıdır. Bu saaten sonra daire başkanı hangi işi kim yapacak derse o kişi o işi yapar.

IoC ile tüm bu bağımlılıklar bir Container içerisine atılır. İhtiyaç halinde bu Container içerisinden alınır. Bağımlılıklar devam eder ama bu bağımlılıkların kontrolü elde edilir.



Dependecy Injection kullanılarak enjekte edilecek olan tüm nesnelere IoC Container denilen bir sınıfta tutulut.

İhtiyaç halinde bu nesneler Container'dan çağırılarak elde edilir.

Asp.Net Core'da IoC Yapılanması

Asp.Net Core kendi içerisinde IoC yapılanması ile gelen bir mimaridir.

Bunun dışında .Net uygulamalarında IoC yapılanmasını sağlayan third party framework'ler mevcuttur:

- Structuremap
- AutoFac
- Ninject

Aynı zamanda Asp.Net Core mimarisi bu third party kütüphaneler kadar yetenekli olmasa da içerisinde dahili gelen bir IoC Container modülü bulunur.

Built-in IoC Continer

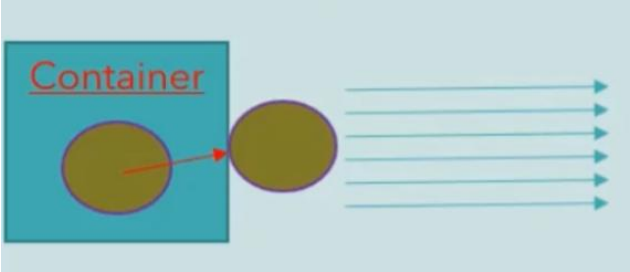
Built-in IoC Container, içerisine koyulacak nesneleri üç farklı davranış ile alabilmektedir.

- Singlton
- Scoped

- Transient

Singleton

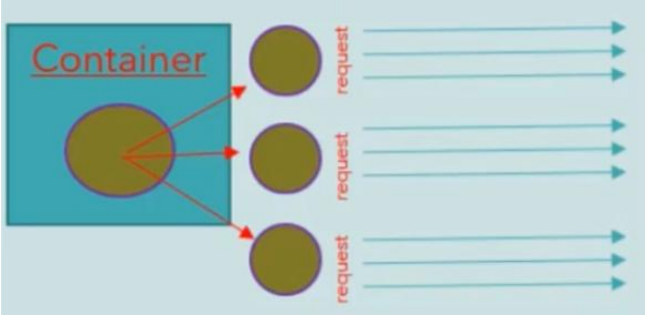
Uygulama bazlı tekil nesne oluştruan ve yapılan tüm taleplerde o nesneyi döndüren bir davranış.



Herhangi bir zamanda nesne değişirse yapılan tüm isteklerde değişmiş nesne gönderilir. Diyelim ki 1 milyon istek geldi, bu bir milyon istek için sadece 1 nesne oluşturur ve o nesneyi gönderir hepsine.

Scoped

Her Request için bir nesne üretilir ve o Request Pipeline'ında olan tüm isteklere o nesneyi gönderir. 1 milyon istek gelirse her istek 1 tane toplam 1 milyon nesne oluşturur. Gelen talep sayısına göre nesneleri gönderir.



Transient

Her Request'in her talebine karşılık bir nesne üretir ve gönderir. En maliyetlisidr. 1 milyon istek var ve her istek içerisinde 10 tane talep var. 10 x 1 milyon nesne üretir.

Built-in IoC(IServiceCollection) Çalışma Sistemi

.Net Core mimarisi içerisinde gelen Ioc yapısı IServiceController ile kullanılır.

IServiceController'dan bir container üretilir ve bu container'a gerekli sınıflar kayıt edilir.

İhtiyaç duyulduğu zaman bu container içerisinden gerekli sınıflar çekilebilir.

```
IServiceCollection services = new ServiceCollection();
services.Add(new ServiceDescriptor(typeof(ConsoleLog), new ConsoleLog()));
services.Add(new ServiceDescriptor(typeof(TextLog), new TextLog())); // Service ekleme


ServiceProvider provider = services.BuildServiceProvider(); // Nesneyi sağlayaak olan somut container
provider.GetService<ConsoleLog>();
provider.GetService<TextLog>(); // ilgili sınıfı getirir.
```

Bu şekilde ekleme yapılınca Default olarak Singleton olarak çalışır.

Asp.Net Core IoC(IServiceCollection) Kullanımı(Add fonksiyonu ile)

Asp.Net Core mimarisinde işleri kolaylaştırmak için bu Container direkt olarak program.cs içerisinde bulunur. Buradan ilgili Container'a eklenen service'ler(sınıflar) tüm uygulamada "new"lenmeden kullanılıniliyor.

Eskiden bu iş StartUp.cs dosyasında yapılıyordu. ConfigureServices fonksiyonu vardı, bu fonksiyona parametre olarak IServiceCollection türünden bir services parametresi alıyordu. Bu parametre üzerinden servisler "add()" veya benzeri fonksiyonlar yardımı ile eklenebiliyordu.

```
public void ConfigureServices(IServiceCollection services)
{
    services.Add(new ServiceDescriptor(typeof(ConsoleLog), new ConsoleLog()));
    services.Add(new ServiceDescriptor(typeof(TextLog), new TextLog()));
    services.AddControllersWithViews();
}
```

.Net 6 ve sonraki sürümlerde ise Program.cs içerisinde Builder.Services.Add() fonksiyonu ile aynı işlemler yapılabiliyor.

```
// Add services to the container.
builder.Services.AddControllersWithViews();
builder.Services.Add(new ServiceDescriptor(typeof(ConsoleLog), new ConsoleLog()));
builder.Services.Add(new ServiceDescriptor(typeof(TextLog), new TextLog()));
```

Buradaki davranışı(Singleton, Scoped, Transient) değiştirmek için ServiceDescriptor fonksiyonunun 3. Parametresi overload edilir. 3. Paramtere olarak ServiceLifetime üzerinden istenilen davranış seçilir.

```
builder.Services.Add(new ServiceDescriptor(typeof(TextLog), new TextLog(),ServiceLifetime.Transient));
```

Burada davranış belirlemek çok uğraştırıcı olduğu için Add fonksiyonu kullanmak yerine davranışa özel AddSingleton, AddScoped, AddTransient gibi davranışa özel Add fonksiyonları kullanılabilir.

```
builder.Services.AddSingleton<ConsoleLog>();
builder.Services.AddScoped<ConsoleLog>();
builder.Services.AddTransient<ConsoleLog>();
```

Bu sayede çok daha kolay ve hızlı bir şekilde Register yapılabilir.

Container’a Nesne Eklerken Nesnenin Consturactor’una Parametre Verme

Eğer Container’a eklenen nesne Consturactor’ında bir parametre bekliyorsa sistem patlar. Böyle bir durumda Container’a nesne eklerken parantez içerisinde istenilen parametreler verilebilir. Parametre içerisinde LinQ yardımı ile örnek bir nesne istenilen parametre ile oluşturulur.

```
builder.Services.AddSingleton<ConsoleLog>(p => new ConsoleLog(5));
```

Bu kullanım tüm davranışlar için geçerlidir.

Interface ile Register Etme

Ortak türleri ortak bir interface altında toplayıp öyle Register etmek daha kolay ve doğru bir yoldur.

Tek tek sınıfları Register etmektense o sınıfın kalıtıldığı interface’i Register etmek daha kolay.

```
builder.Services.AddScoped<ILog>(p => new ConsoleLog(5));
```

Veya generic olarak

```
builder.Services.AddScoped<ILog, ConsoleLog>();
```

Fakat bu kullanımda ilgili nesnenin Consturactor’unun parametre almaması gerekir.

Container’a Eklenen Nesnelerin Controller’da Kullanımlası/talep edilmesi(Context)

Controller’ların Consturactor’ları üzerinden bu Container’ların içerisindeki nesneleri erişilebiliyor.

Controller içerisinde Private Readonly ile Container’dan çekilecek türün bir field’I oluşturulur.

Sonrasında ilgili Controller’ın Consturactor’ında Container içerisinden istenilen nesneler patametre olarak çekilir.

Consturactor’da Field’a, Container’dan çekilen nesne atılır.

Sonrasında bu nesne Controller içerisinde kullanılır.

Field global olarak oluşturulduğu için Controller içerisinde heryerden kullanılabilir.

```
public class LogController : Controller
{
    private readonly ILog _log;
    0 references
    public LogController(ILog log)
    {
        _log = log;
    }
```

Artık Controller’da Container içerisinden gelen nesnenlere erişilebilir.

Mesela Action içerisinde _log üzerinden Log() fonksiyonu çağırılabilir.

Fakat burada önemli bir durum var. Program.cs içerisinde Ilog üzerinden ConsoleLog() kayıt edildiği için Log() fonksiyonu çağırıldığında ConsoleLog içerisindeki Log() fonksiyonu çağırılacak.

```
builder.Services.AddScoped<ILog, ConsoleLog>();
```

Eğere ilerde bir gün ihtiyaç değişirse, Program.cs içerisindeki bu kısımda TextLog yazmak yeterli olacaktır.

Container’a Eklenen Nesnelerin Action’da Kullanımlası/Talep edilmesi

Action bazlı Dependecy Injection yapmak için yani direkt olarak Action üzerinden nesneleri talep etmek için Action’a, “[FromServices]” Attribute’u ile ilgili nesne verilir.

Container’a Eklenen Nesnelerin View’de Kullanımlası/Talep edilmesi

Controller veya Action yerine direkt View üzerinden de ilgili nesneler talep edilebilir.

View içerisinde inject keyword’ü ile ilgili servisin yolu ve adı verilir ve bu ad üzerinden View içerisinde istenilen nesneler çağırılabilir.

```
@inject P10_Dependency_Injection.Services.ILog log

@{
    log.Log();
}
```